Testing

# 03: High-Performance Buses

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
*Indiana University*

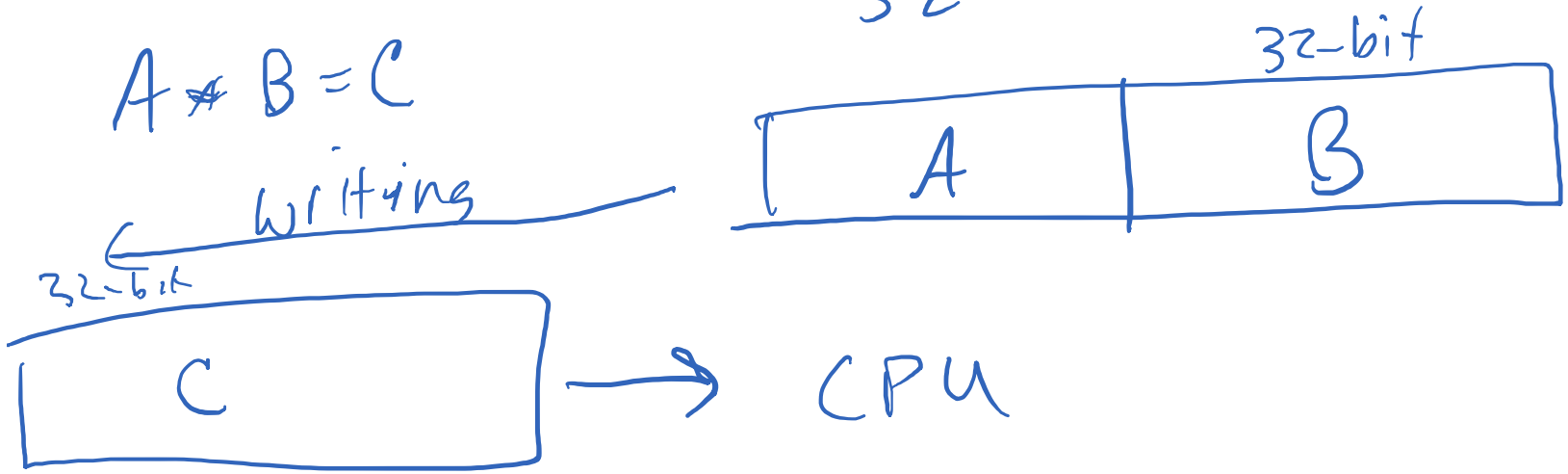
# Let's talk P2

Tutorial Goal: 16-bit * 16-bit multiply

32-bit

32-bit bus

A * B = C



# Let's talk P2

32-bit × 32-bit = 64 bit

A × B = C

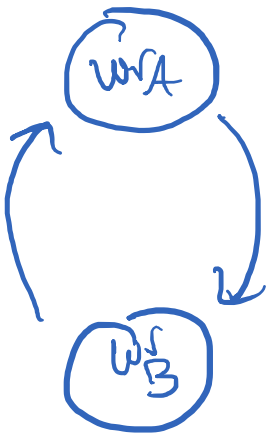
0x3200 → A

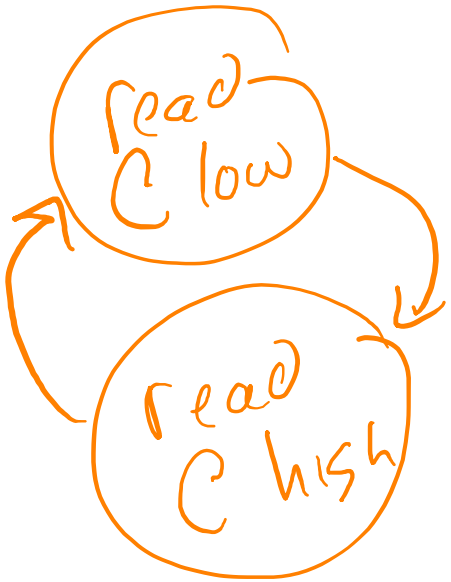
~~0x3204 → B~~

~~0x3208 → C~~ (low 32 bits)

~~0x320C → C~~ (high bits)

write A
write B



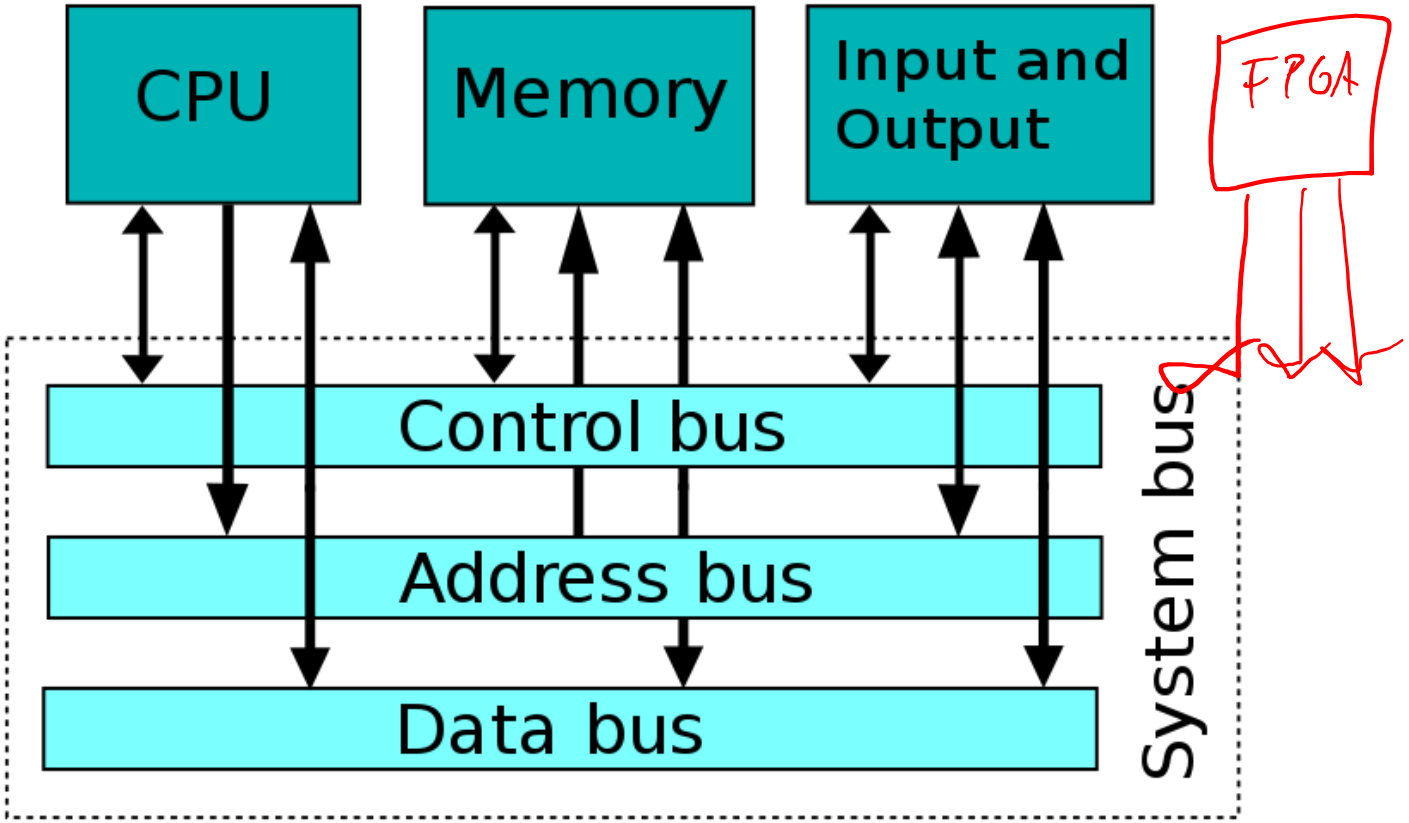
# Other Announcements

- Ethan's Office Hours schedule

| | |
|---|---|
| Wednesday | 5.30 – 8:00 pm |
| Thursday | 2.30 – 4:00 pm |

- New FPGA boxes are ordered

# The System Bus




mem

| | |
|---|---|
| 0x9 | ~~01~~05 |
| 0xb | ~~02~~06 |
| 0xC | 03 |
| 0xd | 04 |
| 0xe | |
| 0xf | |

0x01020304

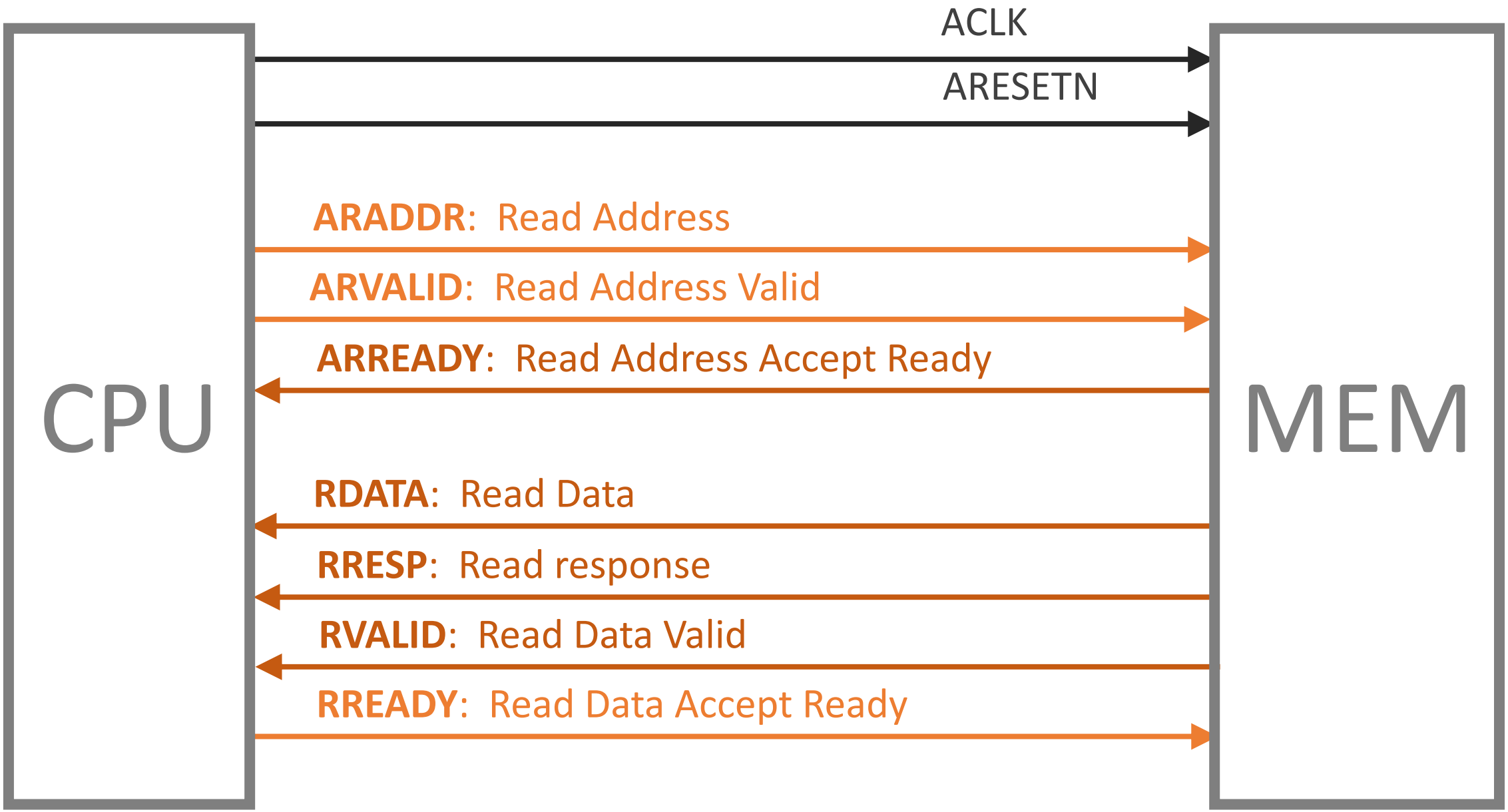
0x a

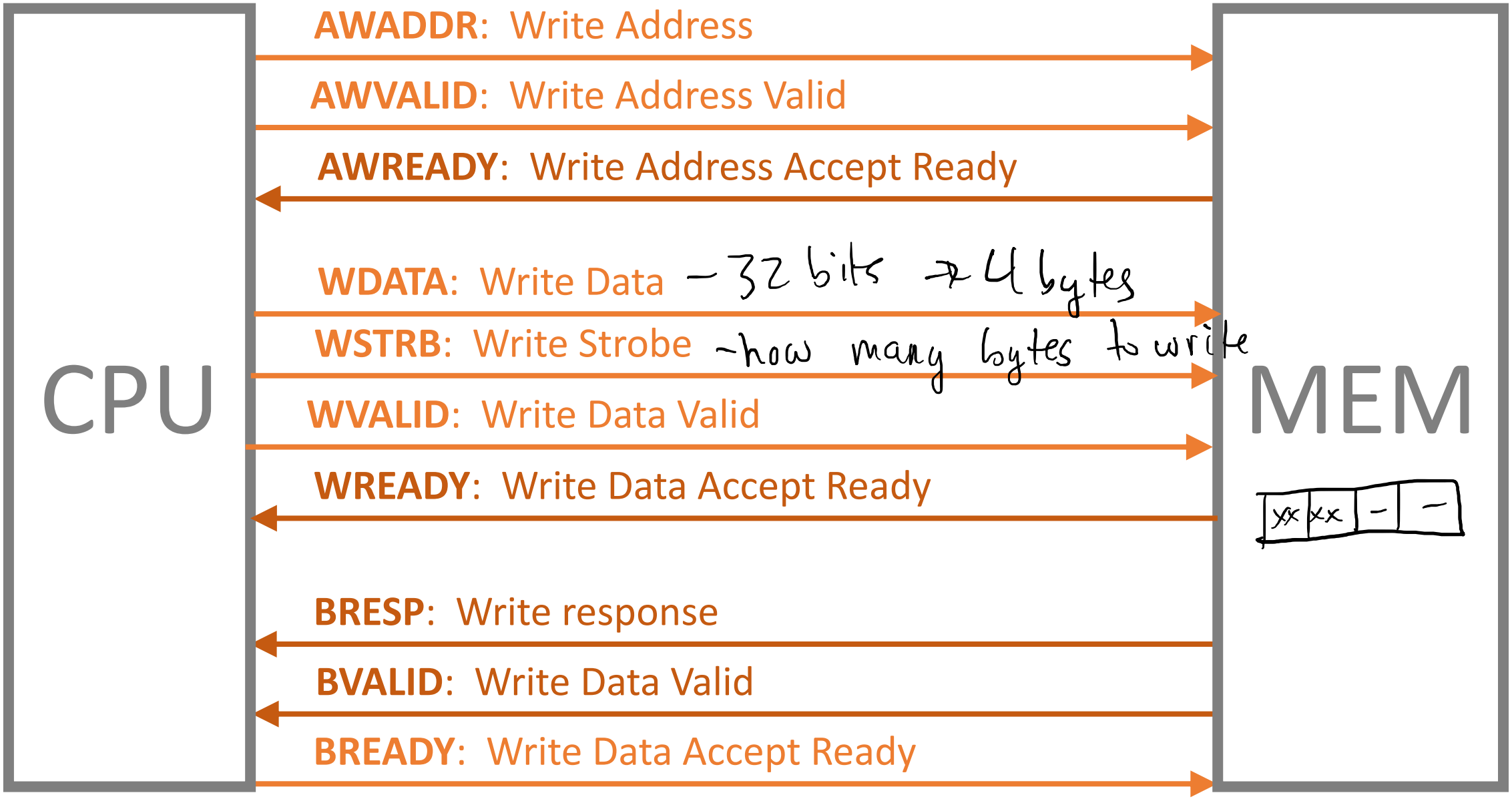
0xa → 0x 0506 xxxx

* wrong endianness

# ARM AXI Bus

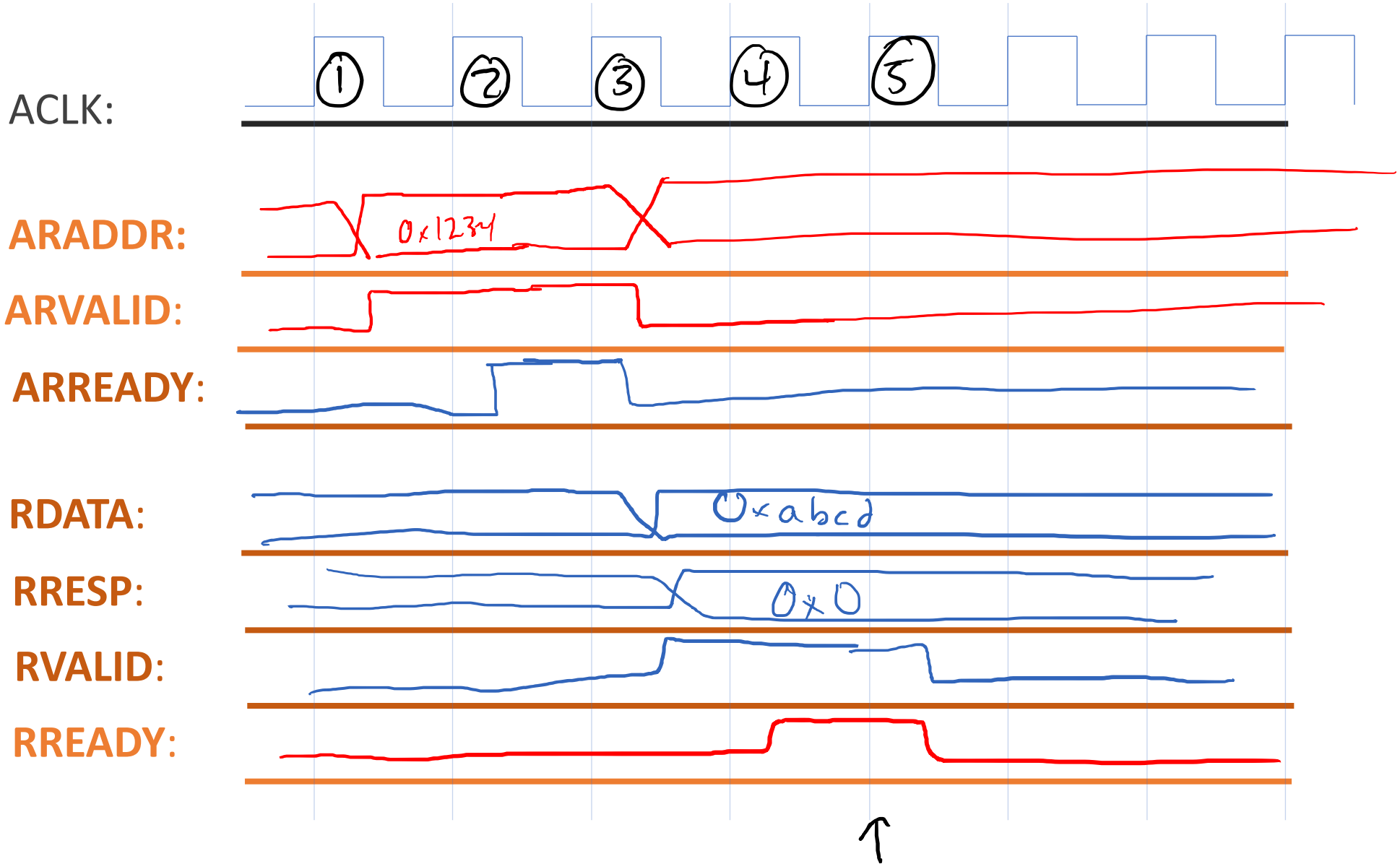
- “Advanced eXtensible Interface” Bus Version 4, “AXI4”
- Three Variants
  - AXI4: Fast but complicated; Memory-mapped
  - AXI4 Lite: Slow but simple; Memory-mapped
  - AXI4 Stream: Fast and simple; Not memory-mapped


ACLK
ARESETN
ARADDR: Read Address
ARVALID: Read Address Valid
ARREADY: Read Address Accept Ready
CPU
MEM
RDATA: Read Data
RRESP: Read response
RVALID: Read Data Valid
RREADY: Read Data Accept Ready

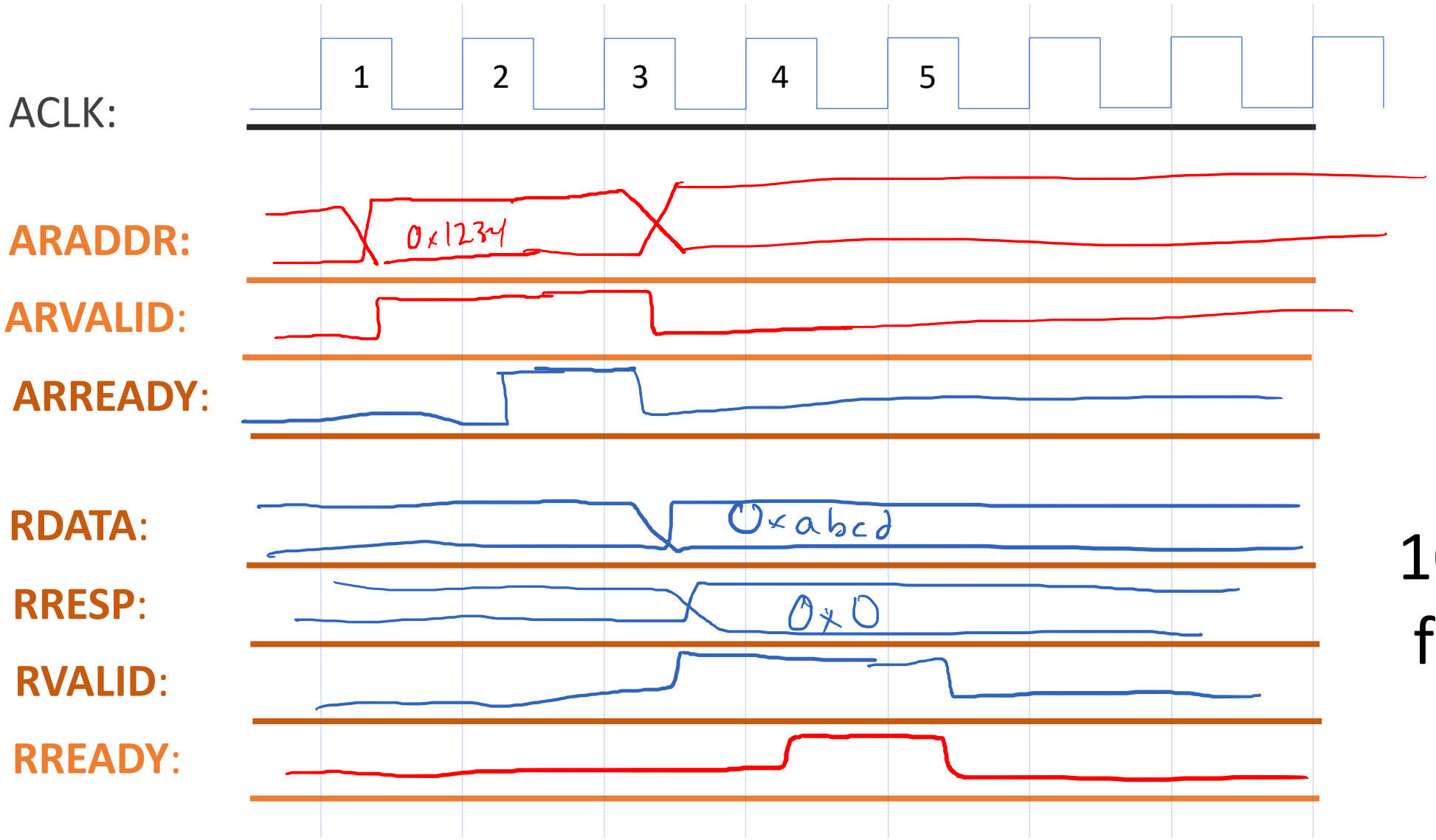



ACLK and ARESETN not shown

# How long does this take?



# How long does this take?




5 cycles per read

10 cycles for 2 reads

# High-Performance Bus Ideas

- Make single transaction faster

# AXI Handshake Speedup

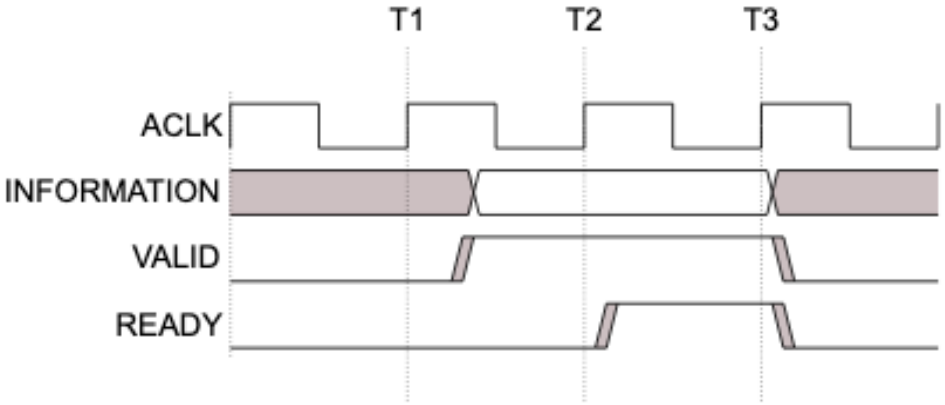



**Figure A3-2 VALID before READY handshake**

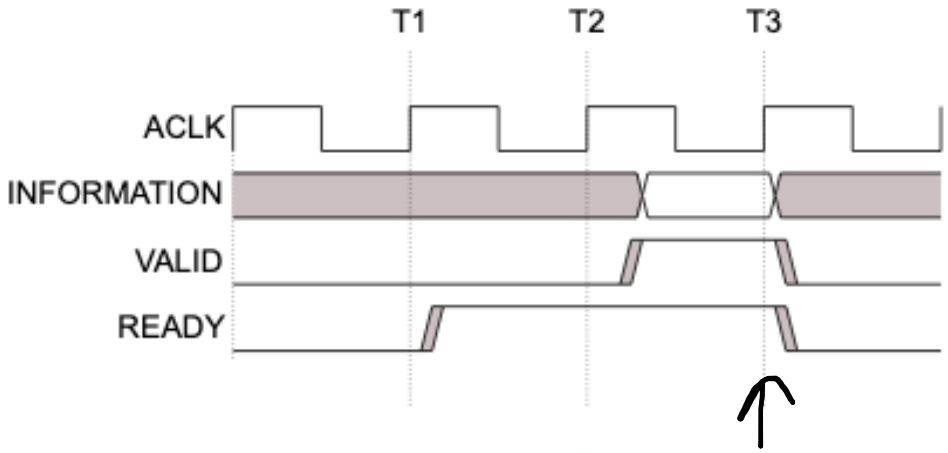



**Figure A3-3 READY before VALID handshake**

- Both are valid
- Figure A3-3 is faster

# What happens here?

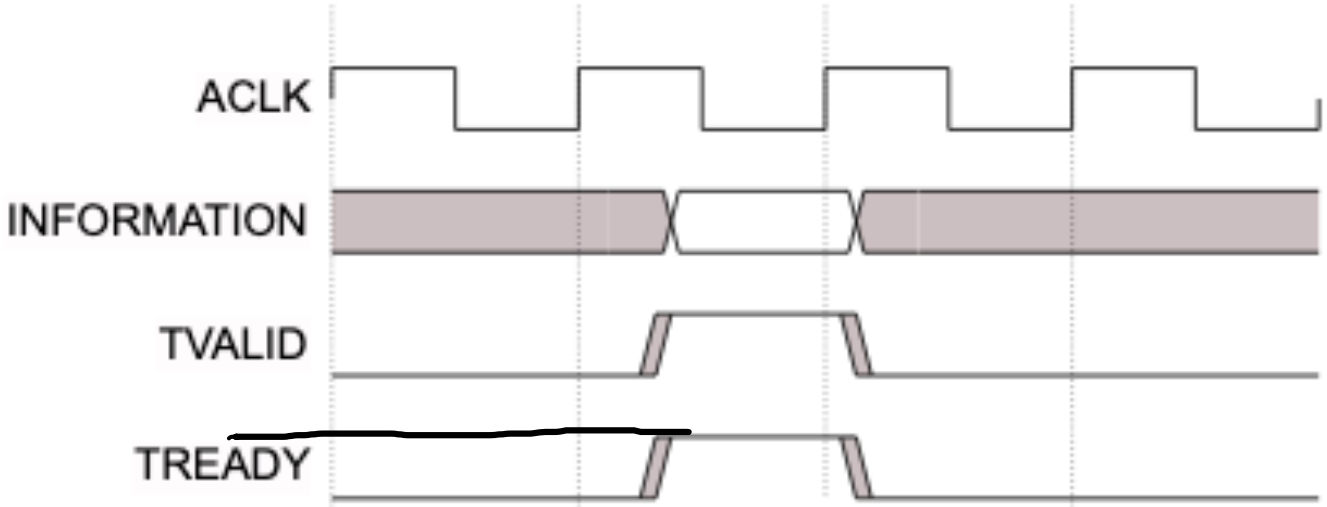
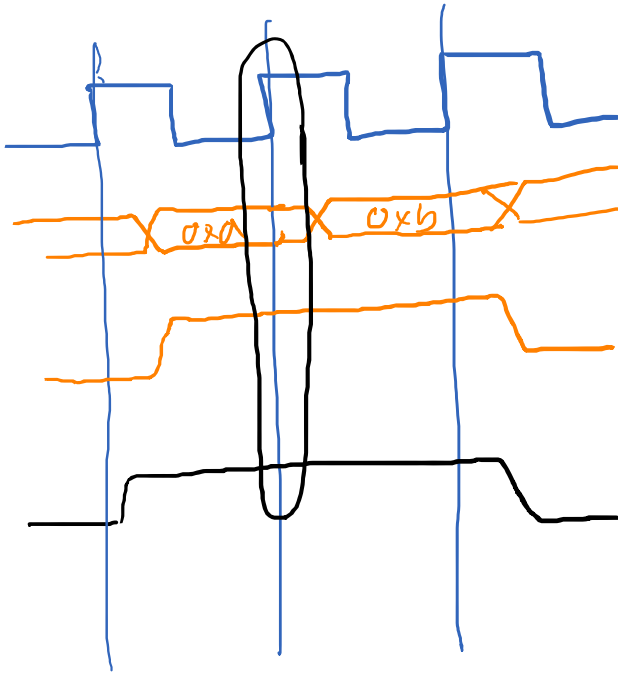



**Figure 2-3 TVALID with TREADY handshake**

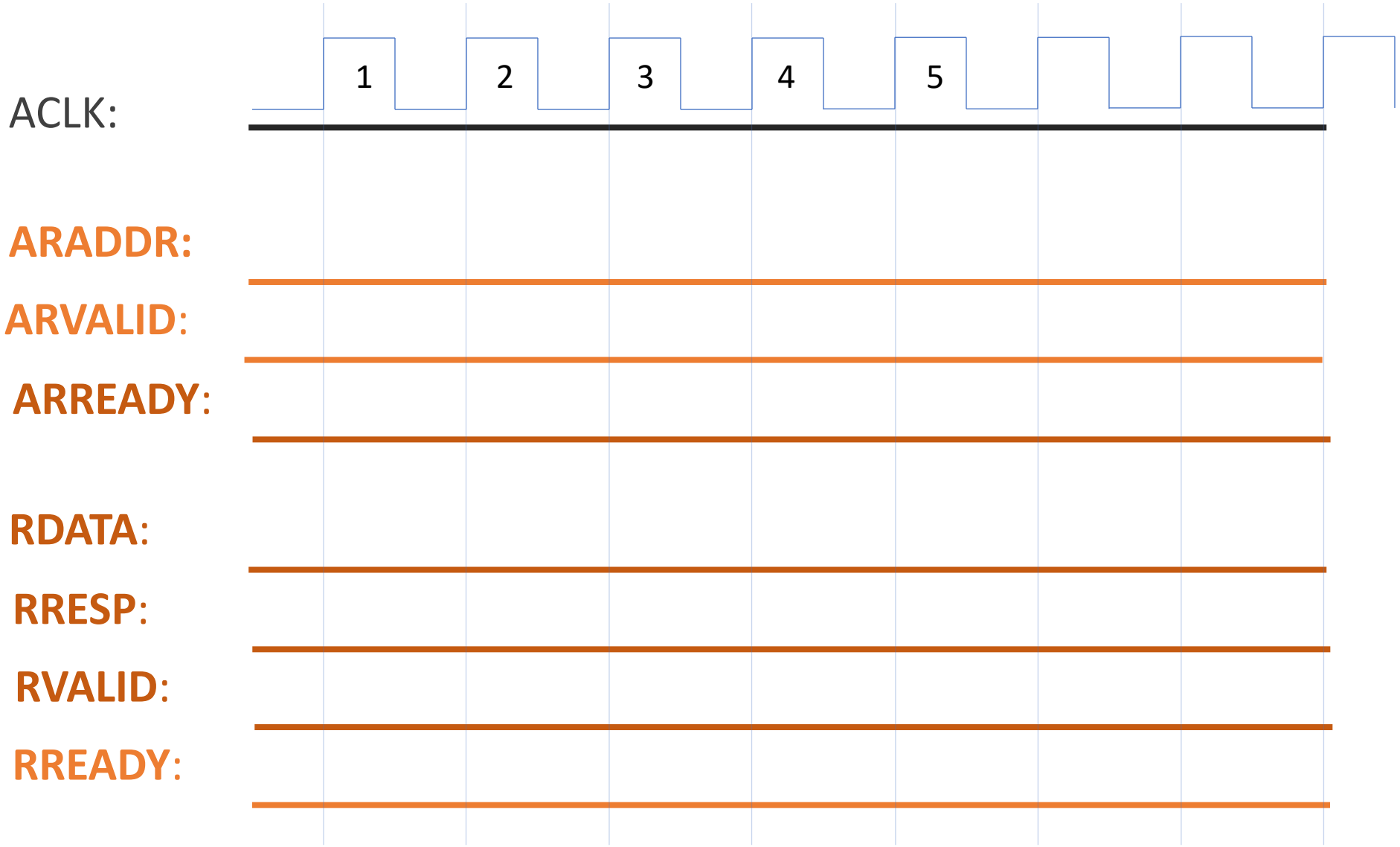
Goal:




- wont work for AXI Lite Reads (maybe writes?)

# What can we do to make this faster?

assume ARESETN = 1

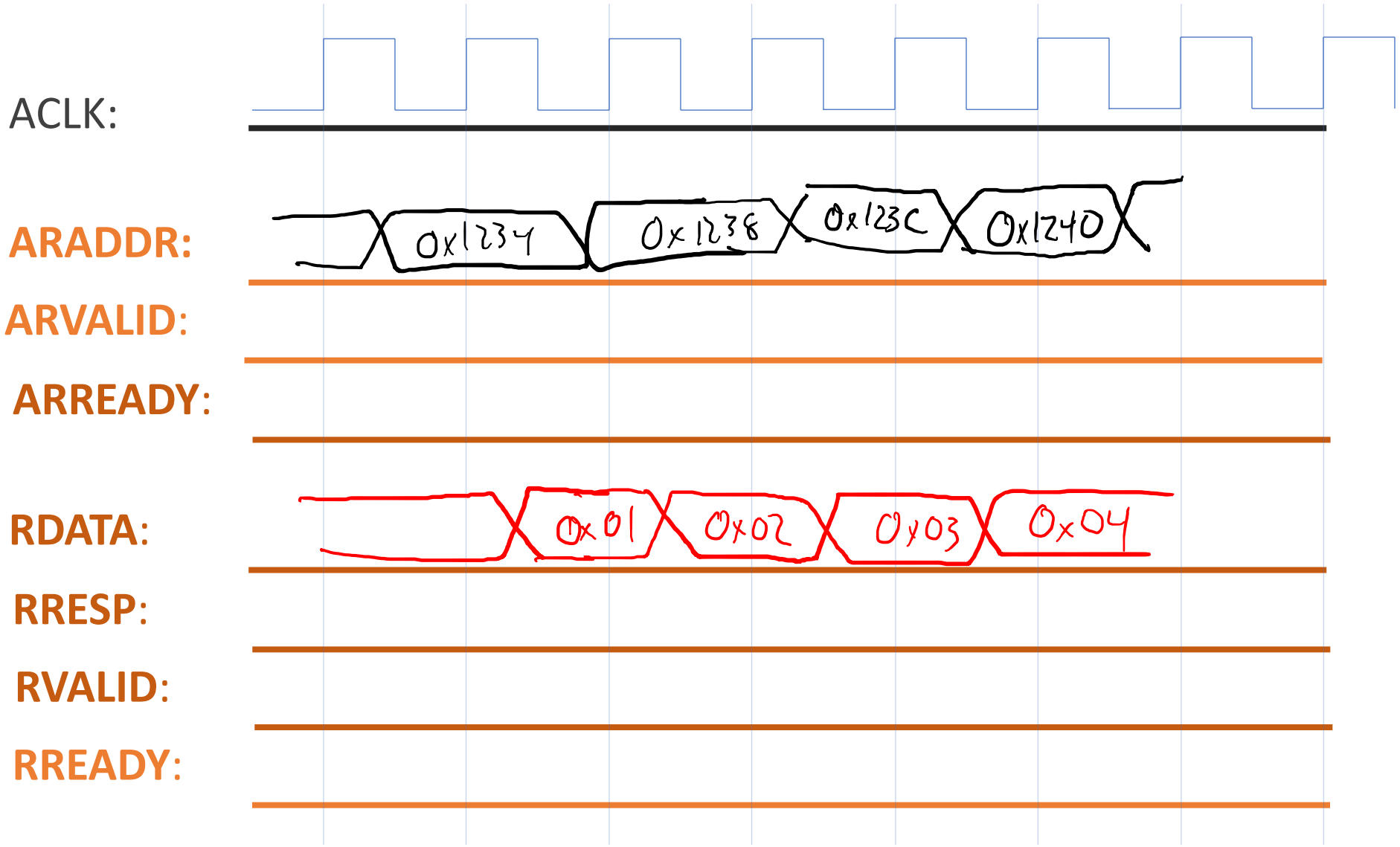


# High-Performance Bus Ideas

- Make single transaction faster
- Overlap multiple transactions

# Can we load 0x1234 and 0x1238?

assume ARESETN = 1



# Burst Transactions

- When a device is transmitting data repeatedly without going through all the steps required to transmit each piece of data in a separate transaction

# Burst Transaction

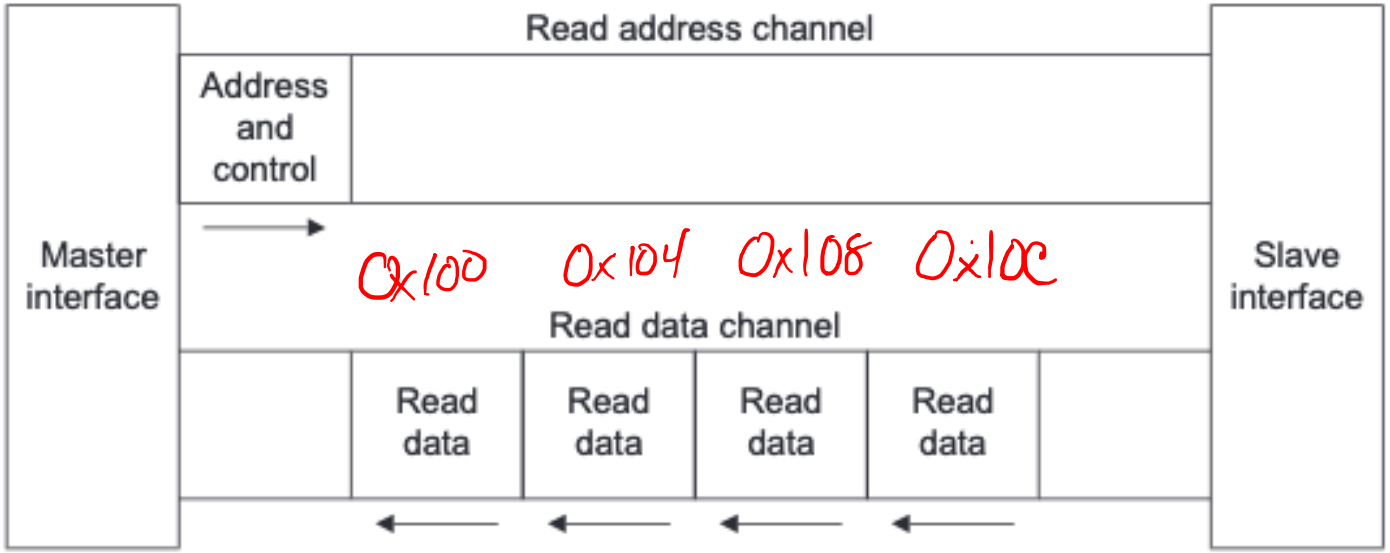
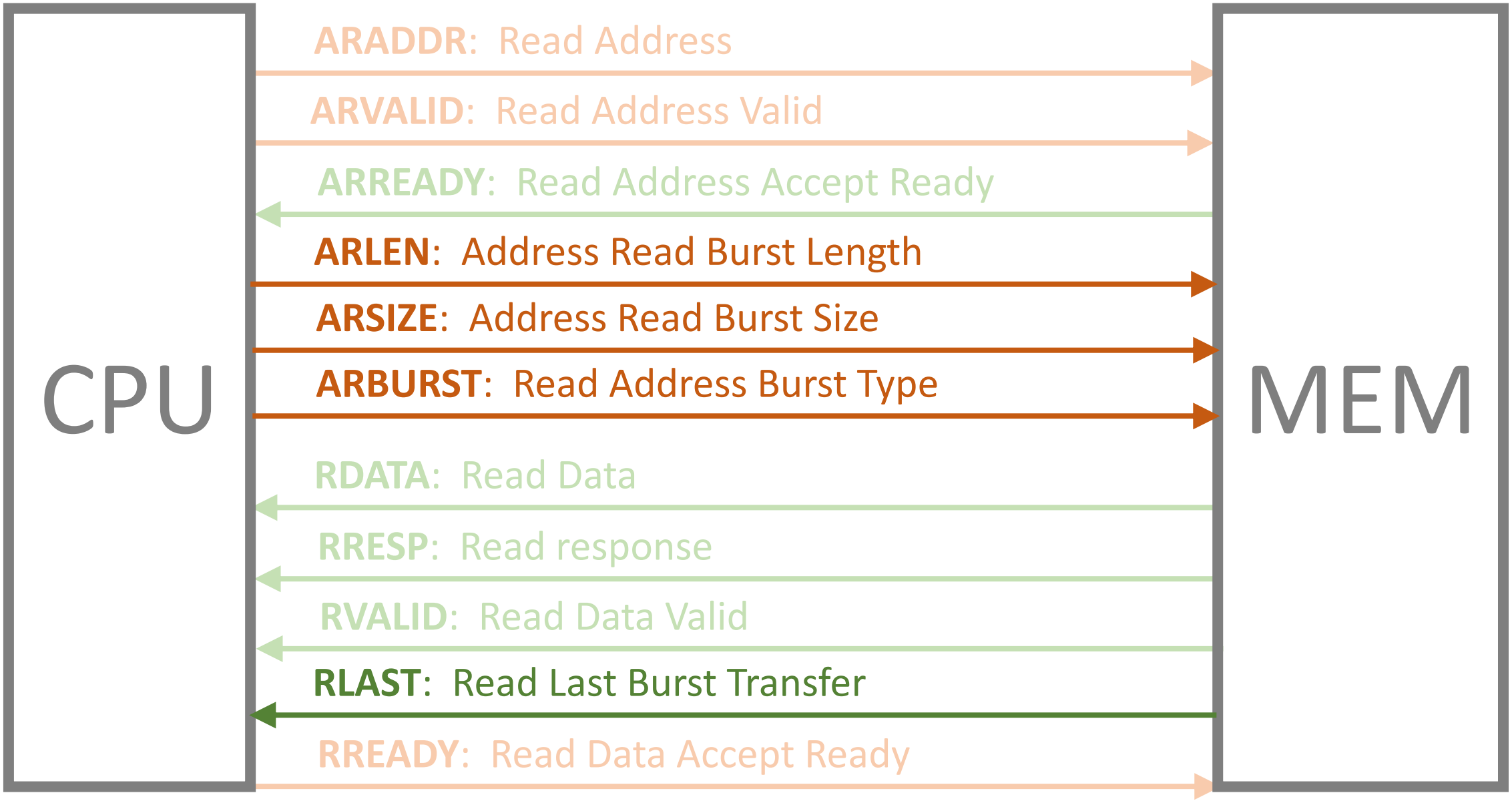



**Figure 1-1 Channel architecture of reads**


CPU
MEM
ARADDR: Read Address
ARVALID: Read Address Valid
ARREADY: Read Address Accept Ready
ARLEN: Address Read Burst Length
ARSIZE: Address Read Burst Size
ARBURST: Read Address Burst Type
RDATA: Read Data
RRESP: Read response
RVALID: Read Data Valid
RLAST: Read Last Burst Transfer
RREADY: Read Data Accept Ready

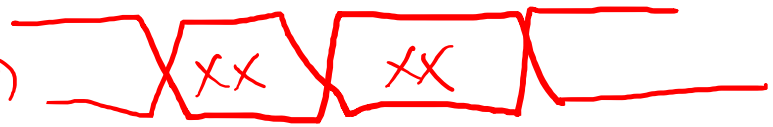
# What do the new signals do?

**ARLEN**: Address Read Burst Length

How many bursts should occur? (+1)

Burst_Length = ARLEN[7:0] + 1

Burst Length = 2 →




⇒ ARLEN = 1

Burst len = 4

# What do the new signals do?

**ARLEN**: Address Read Burst Length

How many bursts should occur? (+1)

**ARSIZE**: Address Read Burst Size

How many bytes should be in each burst?

$2^n$ ~~Bits~~ Bytes / Transfer

**Table A3-2 Burst size encoding**

| AxSIZE[2:0] | Bytes in transfer |
|---|---|
| 0b000 | 1 |
| 0b001 | 2 |
| 0b010 | 4 |
| 0b011 | 8 |
| 0b100 | 16 |
| 0b101 | 32 |
| 0b110 | 64 |
| 0b111 | 128 |

# What do the new signals do?

**ARLEN**: Address Read Burst Length

How many bursts should occur? (+1)

**ARSIZE**: Address Read Burst Size

How many bytes should be in each burst?

**ARBURST**: Read Address Burst Type

Are the addresses incrementing, or repeating?

FIXED: The address is the same for every transfer ( Next Address = Address )

INCR: The address for each transfer is an increment of previous transfer

( Next Address = Address + 0x4 )

**Table A3-3 Burst type encoding**

| AxBURST[1:0] | Burst type |
|---|---|
| 0b00 | FIXED |
| 0b01 | INCR |
| 0b10 | WRAP |
| 0b11 | Reserved |

# What do the new signals do?

**ARLEN**: Address Read Burst Length

How many bursts should occur? (+1)

**ARSIZE**: Address Read Burst Size

How many bytes should be in each burst?

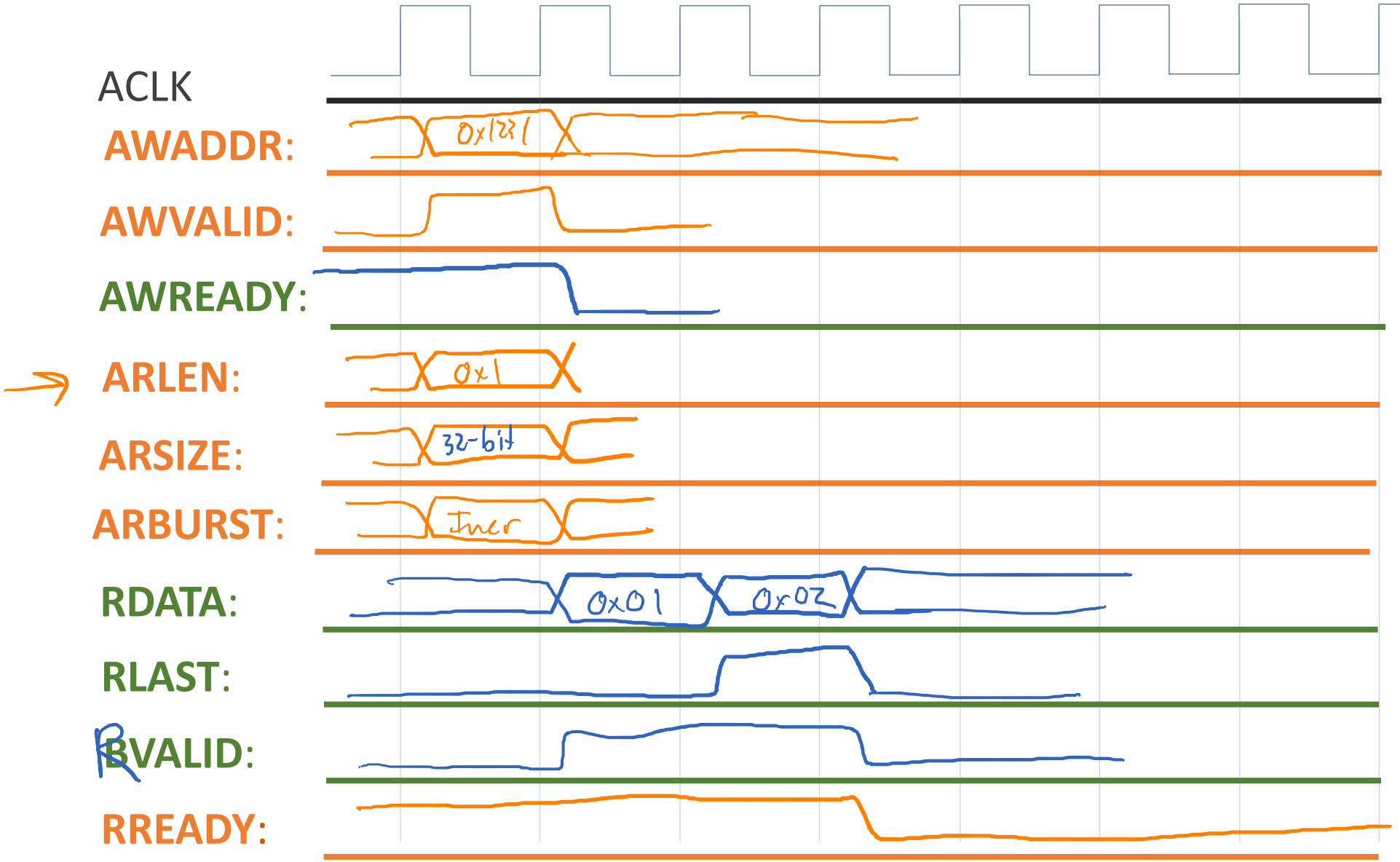
**ARBURST**: Read Address Burst Type

Are the addresses incrementing, or repeating?

**RLAST**: Read Last Burst Transfer

Are we done yet?

# Reading 0x1234 and 0x1238




ARESETN and RRESP omitted

# Read Burst Example

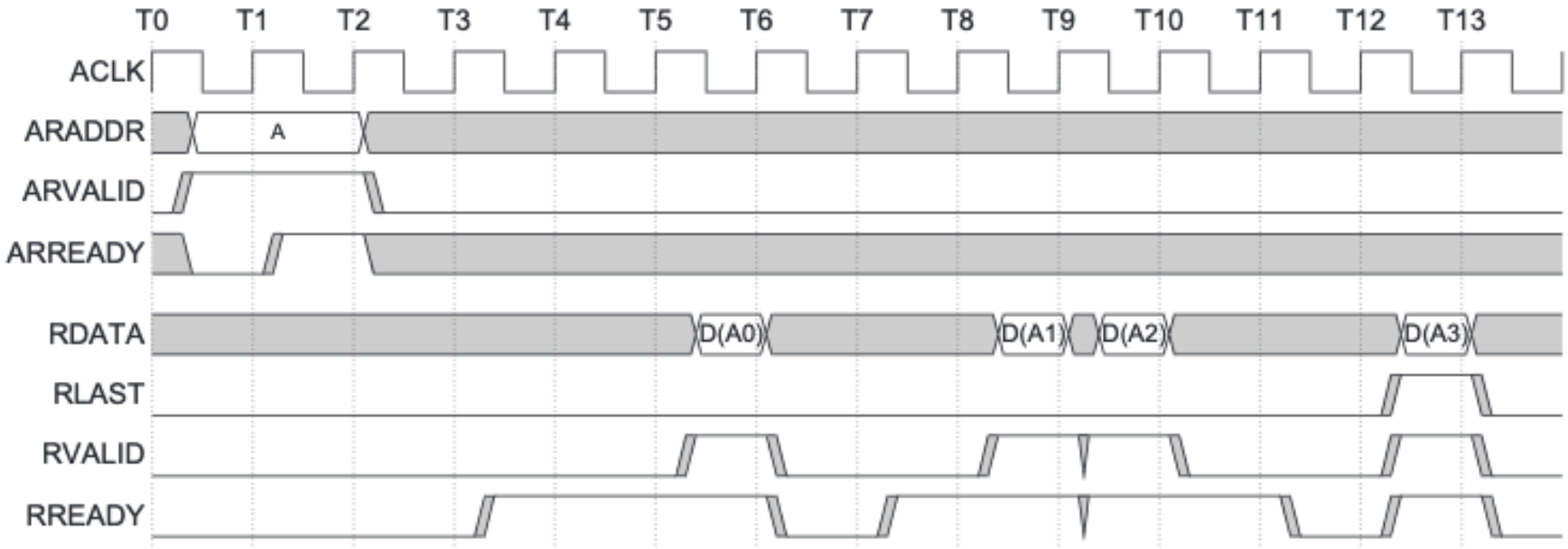



**Figure 1-4 Read burst**

**Note**

The master also drives a set of control signals showing the length and type of the burst, but these signals are omitted from the figure for clarity.

# Overlapping Read Burst Transactions

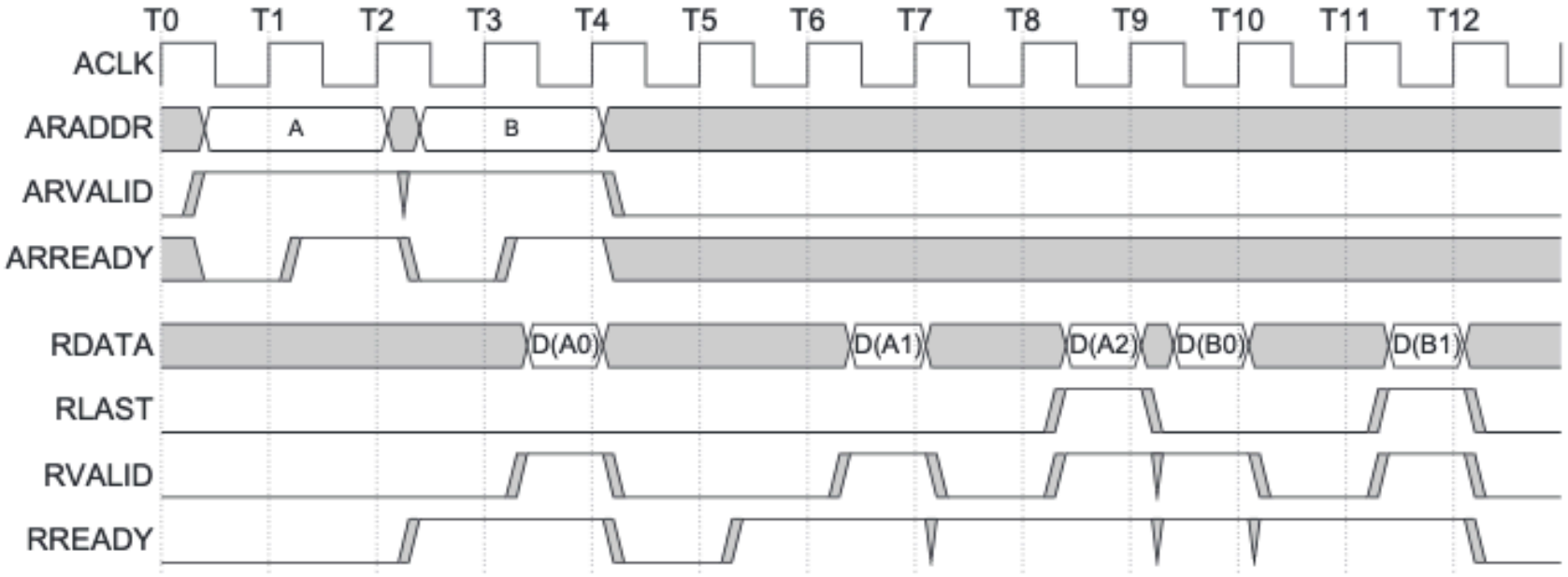
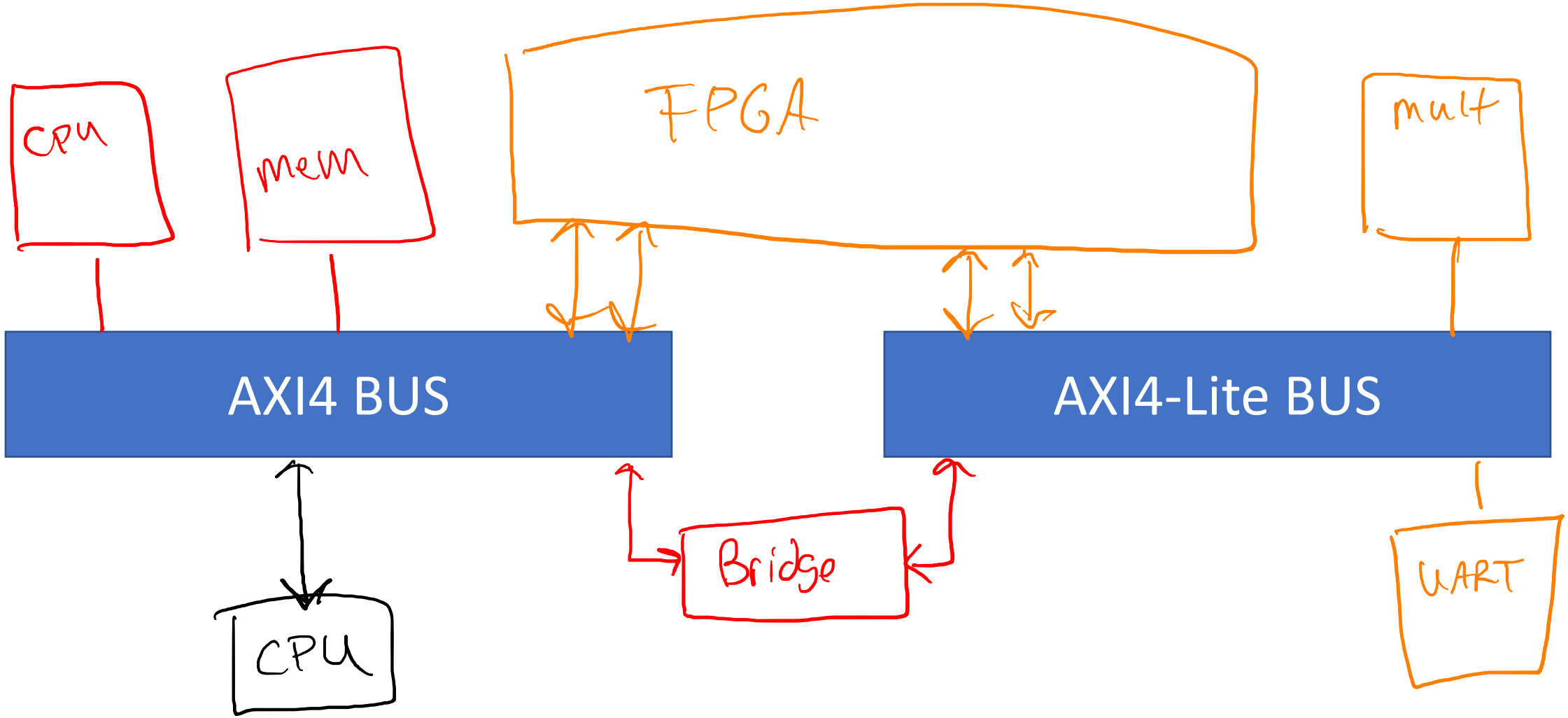
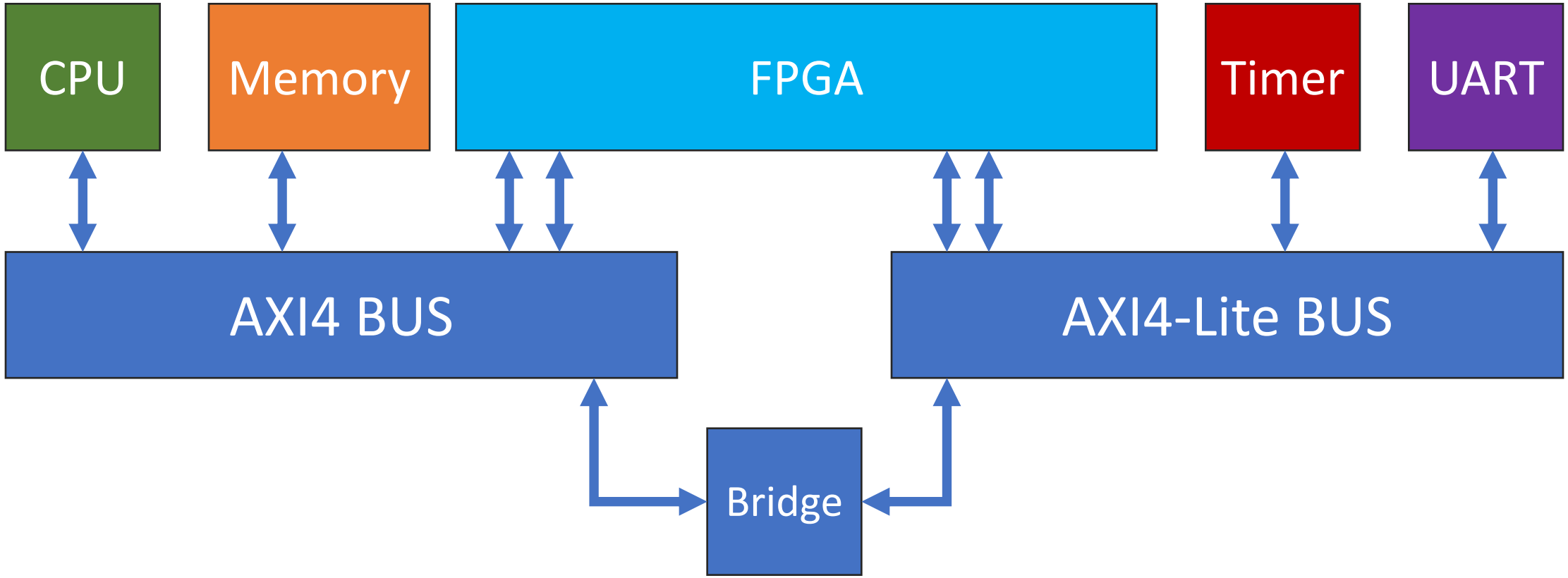


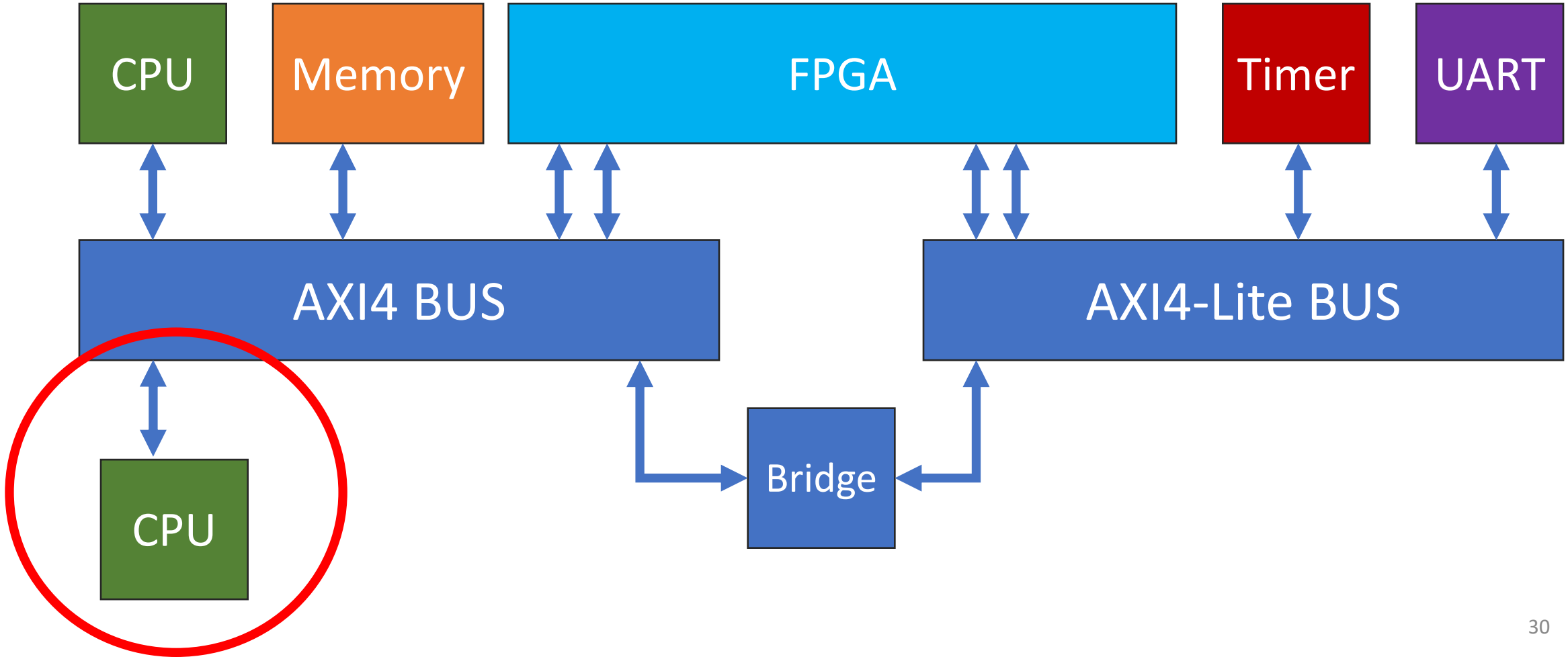

**Figure 1-5 Overlapping read bursts**


CPU
mem
FPGA
mult
AXI4 BUS
AXI4-Lite BUS
Bridge
CPU
UART

# System Architecture



# Next Time: Multiple Masters



# References

- Zynq Book, Chapter 19 “AXI Interfacing”
- Practical Introduction to Hardware/Software Codesign
  - Chapter 10
- AMBA AXI Protocol v1.0
  - http://mazsola.iit.uni-miskolc.hu/~drdani/docs_arm/AMBAaxi.pdf

# 03: High-Performance Buses

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
*Indiana University*